Architecture and Design of an Edge GPU System for Autonomous Perception of Mobile Robots in Precision Agriculture

1st Stanislav Čeman
School of Electrical Engineering
University of Belgrade
Belgrade, Serbia
cs200415d@student.etf.bg.ac.rs

2nd Nikola Ružić
School of Electrical Engineering
University of Belgrade
Belgrade, Serbia
ruzic@etf.rs
0009-0000-8829-6085

3rd Veljko Todić
School of Electrical Engineering
University of Belgrade
Belgrade, Serbia
toveljko@etf.rs
0009-0002-0566-2292

4th Kosta Jovanović
School of Electrical Engineering
University of Belgrade
Belgrade, Serbia
kostaj@etf.rs
0000-0002-9029-4465

***Abstract*—This research explores the development of a sophisticated edge-computing architecture designed for autonomous visual perception tasks of mobile manipulators in precision agriculture. By moving high-intensity computational workloads from remote cloud servers to a localized, GPU-accelerated infrastructure, the system achieves a high degree of responsiveness and reliability, fundamental for real-time robotic systems. The proposed solution integrates an NVIDIA Jetson module with an RBKairos mobile platform, using hardware-specific deep learning optimizations and precise manipulation logic. Experimental validation indicates that this design substantially mitigates temporal instabilities in the control loop and enhances the system energy autonomy by 12%, allowing greater duration of autonomous field operations.**

Keywords: edge perception, robotic manipulation, NVIDIA Jetson, hardware acceleration, precision agriculture.

I. Introduction

The deployment of autonomous robotic systems in modern agriculture represents a paradigm shift toward high-precision management of natural resources. These systems are frequently required to operate in unstructured, dynamic, and remote environments where consistent connectivity to centralized cloud infrastructure is neither guaranteed nor desirable. The inherent latency and potential for network-induced failures in cloud-based robotics pose significant risks to the execution of delicate tasks, such as fruit harvesting or precision weeding, where even a millisecond of delay can result in mechanical collision or damage to the crop.

This paper addresses these critical challenges by adopting the Edge Perception paradigm, which localizes the computational intelligence at the very edge of the sensory network. By designing a specialized GPU-centric architecture, we create a deterministic environment capable of processing complex visual data streams with minimal overhead. Our research focuses on providing a holistic engineering solution that balances the demand for high computational throughput with the rigid energy constraints typical of battery-powered mobile manipulators. The transition to localized processing ensures that the robot maintains operational independence, a prerequisite for deployment in large-scale agricultural fields.

The necessity for robust, latency-free systems in precision smart farming has been well-documented [1]. While cloud robotics has historically offered immense computational power [2], unpredictable network latency remains a critical bottleneck for dynamic physical interactions. Recent advancements have demonstrated that bringing computation closer to the mechanical actuator level is absolutely vital for the stability of autonomous manipulators [3], [4], and especially complex autonomous mobile robots [5]. Additionally, application of hybrid Edge and cloud systems can bring up to 75% savings in energy consumption when processing agentic and visual workloads [6].

Furthermore, building upon previous research focused on the global coordination and mission optimization for multi-robot autonomous crop collection [7], this work addresses the critical prerequisite of localized perception.

II. System Configuration and Hardware Architecture

The experimental platform utilized in this study is a hybrid mobile manipulator, comprising a robust mobile base and a collaborative robotic arm with seven degrees of freedom. To facilitate accurate environmental awareness, the system is integrated with an active stereo-vision depth camera. This sensor provides synchronized RGB and depth information,

ISBN 978-86-6200-074-3 © 2026 AM & ETRAN
https://doi.org/10.69994/13Ic26xxx

This work is licensed under CC BY-NC-ND 4.0 International

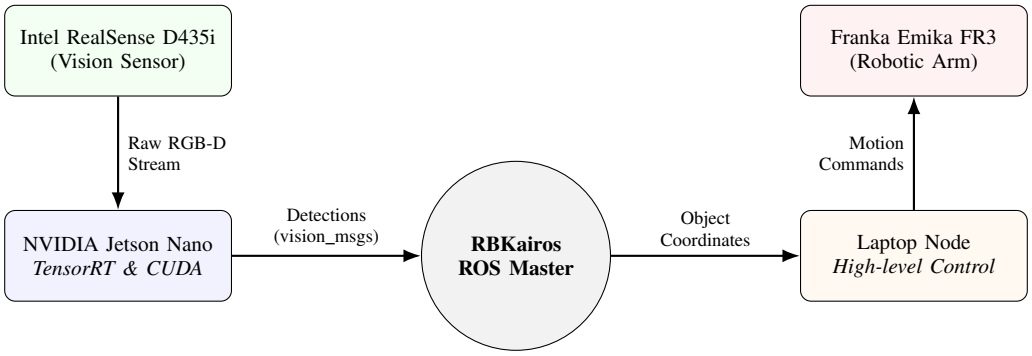


Fig. 1: High-level system workflow and data pipeline architecture. The diagram illustrates the asynchronous data flow from raw sensory input to actuated mechanical response.

which is essential for reconstructing the 3D geometry of the workspace.

The Intel RealSense D435i [8] is used as the main camera for the implemented system. At the heart of the perception system lies an edge GPU module. Rather than utilizing state-of-the-art flagship GPUs, an older-generation NVIDIA Jetson Nano was selected due to its accessibility. The objective is to demonstrate that even highly constrained, legacy hardware can successfully execute complex real-time manipulation tasks when properly optimized.

The system configuration is illustrated in Fig. 1, which outlines the high-level workflow. The RBKairos mobile platform [9] serves as the central ROS Master, orchestrating communication across the local network. Although the RBKairos platform is equipped with an internal processor capable of basic kinematics, an external laptop was integrated as the high-level control node to provide experimental modularity and convenience. The robot's default internal processor possesses limited memory and is not readily modular for rapid prototyping.

Raw visual data from the Intel RealSense camera is streamed directly to the NVIDIA Jetson Nano for edge processing. Within the Jetson node, CUDA-accelerated kernels perform real-time object detection and spatial mapping. The resulting detection coordinates are then published as ROS messages back to the primary control loop. Finally, the high-level control node processes these coordinates to generate and dispatch motion commands to the Franka Emika FR3 arm [10].

III. Design of the Visual Perception Pipeline

The perception pipeline is the primary component responsible for transforming raw visual signals into actionable intelligence. To meet the real-time constraints of mobile manipulation, several layers of optimization were introduced.

A. YOLO Object Detection Framework

The core of the perception engine utilizes the YOLO (You Only Look Once) framework [11], specifically YOLOv5n. Compared to the original Darknet-based YOLO architecture, YOLOv5 introduces significant improvements, including a PyTorch-based ecosystem, a highly efficient CSPDarknet53 backbone, and mosaic data augmentation techniques that improve the detection of small objects. The model's nano version was specifically chosen for this research because its software dependencies, notably Python 3.6 and older PyTorch versions, perfectly match the legacy JetPack SDK and native CUDA libraries supported by the Jetson Nano hardware. Furthermore, its lightweight nature is our primary target for energy efficiency. The network is pre-trained on the standard COCO (Common Objects in Context) dataset.

YOLO treats detection as a single regression problem, mapping pixels directly to bounding box coordinates (x, y, w, h) and class probabilities in one pass. The input image is first resized to 640×640 pixels, and the YOLOv5n model strictly utilizes only the RGB channels to predict the 2D bounding boxes. The grid dimension $S \times S$ is not fixed but serves as a variable parameter dependent on the network stride. Once a target's 2D bounding box is estimated, the depth channel from the RGB-D sensor is mapped within the boundaries of that specific box. This isolates the depth map, allowing the system to calculate the accurate spatial centroid of the object, which is then translated into a precise 3D coordinate for the robot.

As illustrated in Fig. 2, each grid cell simultaneously predicts overlapping bounding boxes and a class probability map. These raw features are finally fused using the Non-Maximum Suppression (NMS) algorithm to yield the final output.

B. Parallel Computing and CUDA Architecture

The core of our perception system leverages the Compute Unified Device Architecture (CUDA) developed by NVIDIA [13]. Unlike traditional sequential processing on a CPU, CUDA allows for massive parallelization by utilizing thousands of small, efficient hardware threads, grouped in blocks inside a grid. In the context of robotic vision, this is particularly beneficial for operations such as image filtering, matrix multiplications in neural networks, and point cloud processing.

By mapping computationally expensive layers onto the GPU's many-core architecture, we achieve the high degree of data parallelism necessary to execute complex deep learning models within the strict timing constraints of autonomous

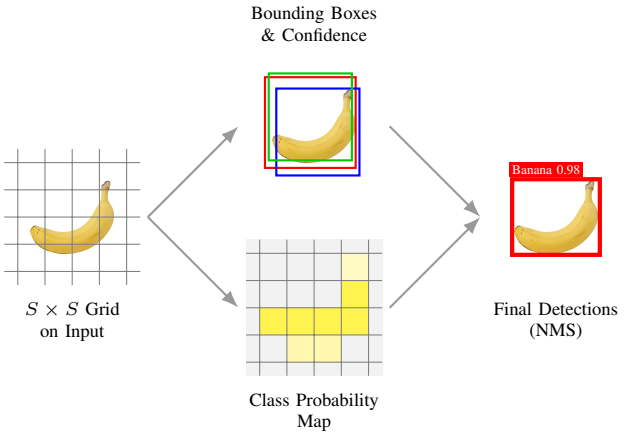


Fig. 2: The YOLO object detection pipeline architecture. The sample image is sourced from Wikimedia Commons [12].

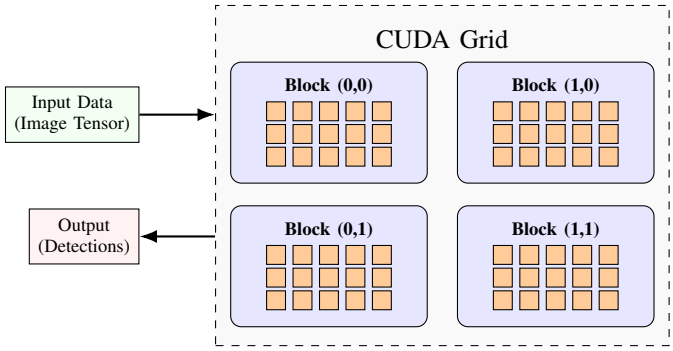


Fig. 3: Architectural hierarchy of the CUDA parallel execution model.

manipulation. A simple parallel structure with 4 blocks inside a grid, each containing 12 threads, is illustrated in Fig. 3.

C. Deep Learning Model Optimization and Quantization

Standard neural network architectures, while accurate, often exhibit prohibitive latency when executed on low-power edge hardware. To overcome this, the detection model was subjected to a hardware-aware optimization process by utilizing the NVIDIA TensorRT Software Development Kit (SDK).

As illustrated in Fig. 4, the optimization engine performs vertical layer fusion. The 'trtexec' utility collapses sequential graph layers in the native YOLOv5n ONNX graph into a unified CUDA kernel. Common operations such as 2D Convolutions, Batch Normalization, and SiLU (Sigmoid-Weighted Linear Unit, also known as Swish) activations are distinct nodes, each requiring separate read/write cycles to the GPU memory. TensorRT fuses these consecutive mathematical operations into a single executable block, effectively eliminating intermediate memory bottlenecks.

Furthermore, the model's weights and activations were quantized from standard 32-bit floating-point (FP32) to 16-bit half-precision (FP16). This strategic reduction in precision allows the system to maximize the throughput of the Jetson Nano's Maxwell architecture, resulting in a dramatic increase

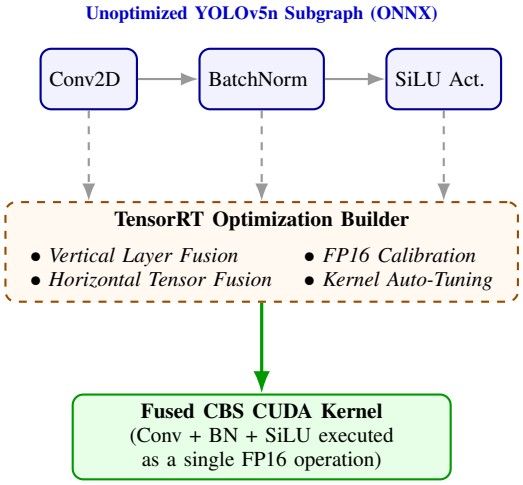


Fig. 4: TensorRT optimization process applied to the YOLOv5n architecture.

in inference speed while preserving the high detection confidence scores required for identifying biological targets in cluttered agricultural scenes.

D. Asynchronous GPU-Centric Pre-processing

To prevent the central processing unit from becoming a performance bottleneck, a custom data flow was established that offloads all image-level operations directly to the GPU. Raw sensory data is ingested into specialized pinned memory buffers, where parallel routines execute spatial rescaling, color space transformations, and normalization. This architectural choice ensures that the external laptop's CPU remains available for high-level decision-making and motion planning, as the Jetson Nano is dedicated exclusively to visual inference. By employing a dual-stream memory architecture, the system is capable of performing inference on one data packet while simultaneously pre-processing the subsequent frame, thereby effectively hiding the computational cost of image preparation and ensuring a jitter-free visual pipeline.

IV. Spatial Calibration and Coordinate Transformations

The translation of a target's position from the image-centric coordinate system to the robot's physical workspace is a foundational requirement for any manipulation task. This requires a precise mathematical model of the spatial relationship between the optical sensor and the mechanical base.

A. Extrinsic Calibration Methodology

The spatial orientation and translation between the camera sensor and the robot's base were determined through an extrinsic calibration methodology based on Zhang's method [14]. This process involved the use of a high-precision checkerboard pattern with a known geometric configuration. By observing the pattern from diverse perspectives within the robot's reach, the system solves a non-linear optimization problem to estimate the camera's pose. This accounts for even the slightest

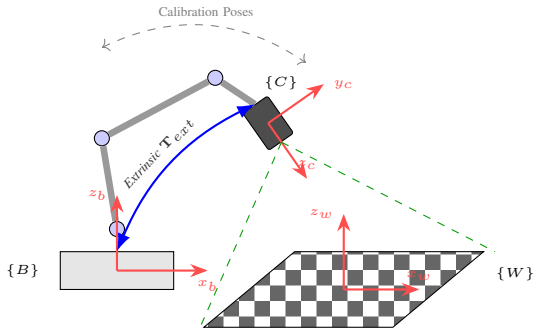


Fig. 5: Extrinsic calibration setup and computation of transformation matrices.

mechanical misalignments in the sensor mounting, providing a transformation matrix that accurately bridges the gap between the visual and mechanical domains.

B. Mathematical Framework for 3D Projection

The mathematical framework defines the direct relationship between a 3D point in the robot's base coordinate system, denoted as $\mathbf{P}_b = [x, y, z]^T$, and its corresponding 2D pixel coordinate $[u, v]^T$ on the image plane. This projection is modeled using the camera's intrinsic calibration matrix $\mathbf{K} \in \mathbb{R}^{3\times3}$ and the extrinsic transformation matrix $[\mathbf{R}|\mathbf{t}] \in \mathbb{R}^{3\times4}$, composed of a rotation matrix $\mathbf{R} \in SO(3)$ and a translation vector $\mathbf{t} \in \mathbb{R}^3$. The forward projection is given by:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}[\mathbf{R}|\mathbf{t}] \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}. \tag{1}$$

For autonomous manipulation, the perception pipeline must perform the inverse operation: de-projecting a localized 2D pixel coordinate back into the 3D mechanical workspace. By isolating the 3D base coordinates from the projection equation, we utilize the inverse intrinsic matrix $\mathbf{K}^{-1}$ and the inverse of the orthogonal rotation matrix ($\mathbf{R}^{-1} = \mathbf{R}^T$). The complete inverse mapping, which translates visual detections into actionable spatial targets, is formulated as:

$$\mathbf{P}_b = \mathbf{R}^{-1} \left(\mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \cdot d - \mathbf{t} \right), \tag{2}$$

where d represents the depth scalar obtained directly from the active stereo sensor.

This rigorous mathematical consistency is what allows the robotic arm to execute high-precision movements toward targets identified in the visual field with sub-centimeter accuracy.

V. Communication Structure and Data Flow (ROS)

The software ecosystem is entirely engineered around ROS 1 Noetic. The RBKairos mobile platform operates as the ROS Master node, orchestrating communication across the local network.



Fig. 6: 4-stage trajectory of fruit grasping action.

A. Manipulation Stability and Execution Logic

Visual sensors operating in field conditions are inherently susceptible to noise caused by changing lighting and atmospheric conditions. To ensure the reliability of the manipulation sequence, we implemented a temporal stabilization algorithm.

This algorithm maintains a sliding window of historical detections and calculates the spatial variance across the samples. A manipulation command is only dispatched when the detection demonstrates a standard deviation below a predefined threshold of 0.015 meters (1.5 cm) across all three spatial axes. Based on the implemented buffer size of 25 frames at a 10 Hz processing rate, this period corresponds to exactly 2.5 seconds of sustained stability. Once the target is stabilized, the system triggers a 4-stage trajectory: transitioning from (1) arm initialization, to (2) approach and grasp phase, followed by (3) return to home pose, and finally (4) a storage phase. Such a modular approach ensures that the robot does not react to false positive detections by the model or transient sensor noise, which is critical for maintaining mechanical longevity. The 4-stage trajectory flow is demonstrated in Fig. 6.

VI. Experimental Results and Energy Analysis

The quantitative performance metrics comparing the external CPU and the edge GPU are summarized in Table I. The performance of the edge GPU architecture was evaluated against a traditional centralized setup utilizing a standard laptop processor (AMD Ryzen 5 4500U). The benchmarks

TABLE I: Quantitative System Performance and Resource Analysis

Parameter	External CPU	Edge GPU
Mean Latency [ms]	84.4	65.1
Frames Per Second [FPS]	11.8	15.4
Temporal Jitter [ms]	44.4	5.4
Average Power Draw [W]	25.0	6.7

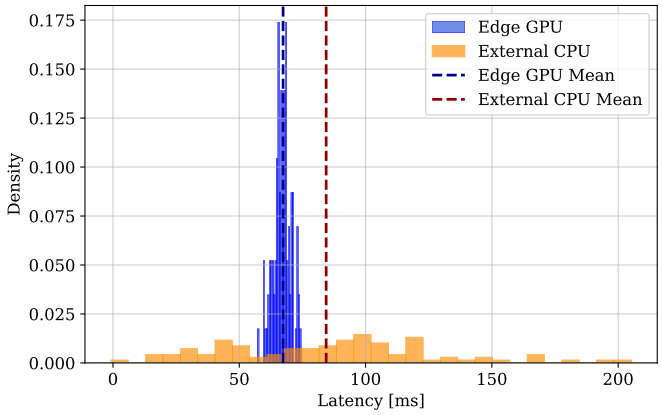


Fig. 7: Latency distribution comparison: Edge GPU vs. External CPU.

focused on latency, frame rate stability, and power consumption.

A. Latency and Temporal Determinism

The extracted data from the NVIDIA tegrastats utility reveals a high degree of temporal determinism, which is a critical requirement for closed-loop robotic manipulation. Fig. 7 illustrates the latency distribution comparison over a sustained operational window. The Edge GPU setup exhibits a tightly bounded variance, with the probability density sharply centered at the mean latency of 65.1 ms. This highly deterministic behavior ensures that visual feedback arrives at the motion planner at predictable intervals, allowing the kinematic algorithms to calculate smooth, continuous arm trajectories.

Conversely, the External CPU setup shows a wide, scattered distribution with significant jitter, frequently experiencing latency spikes well over 100 ms. This unpredictable latency is primarily caused by non-real-time operating system thread scheduling and the sequential processing bottlenecks inherent to traditional CPUs. In a physical robotic system, such temporal inconsistencies are highly detrimental.

B. Power Consumption and Thermal Stability

The localized perception architecture demonstrates vastly superior energy efficiency and environmental resilience. As shown in Fig. 8, the Jetson node operates consistently within a narrow 5–8 W power envelope, seamlessly handling inference spikes without stressing the mobile platform's power delivery network. This contrasts sharply with the external laptop CPU's 25 W average consumption.

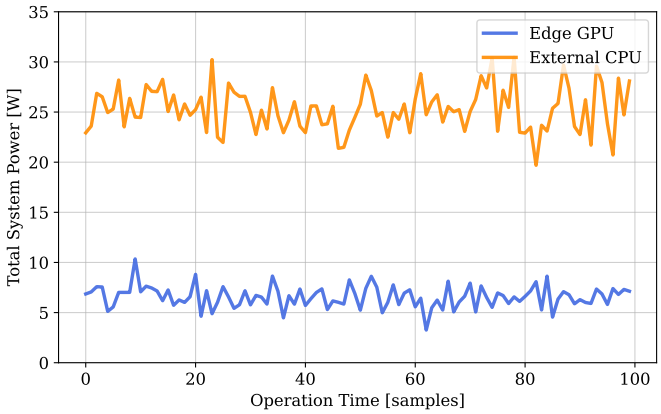


Fig. 8: Real-time power consumption during inference cycles.

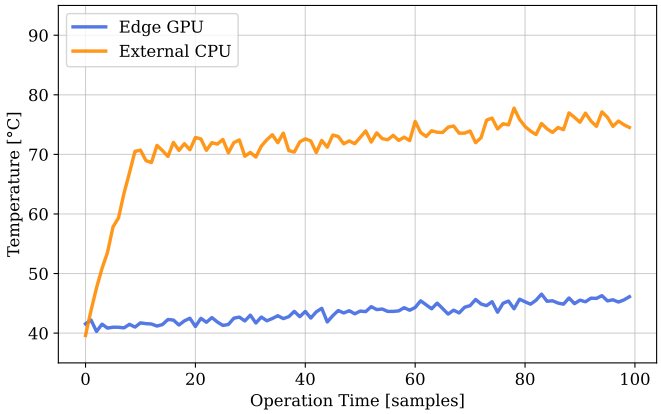


Fig. 9: Thermal profile comparison under sustained computational load.

Beyond mere power draw, the thermal dynamics presented in Fig. 9 highlight a critical operational divergence. The Edge GPU maintains a highly stable temperature profile, plateauing safely below 50°C even under sustained computational load. This thermal stability is vital, as it allows for passive cooling solutions.

In contrast, the External CPU exhibits a rapid temperature escalation, reaching up to 78°C within the first few minutes of operation. While 78°C is within the manufacturer's specified operational limits,it is operating continuously near this upper thermal boundary. The fanless, passively cooled design of the Jetson module completely circumvents this issue.

C. Energy Efficiency

To evaluate the energy efficiency of the proposed architecture, we introduce the *Energy Per Inference (EPI)* metric. The EPI represents the total electrical energy consumed by the computing node to process a single frame through the entire perception pipeline. It is defined as the product of the average power consumption P_{avg} and the mean inference latency L_{mean}:

$$EPI = P_{avg} \cdot L_{mean}, \tag{3}$$

where P_{avg} is measured in Watts (W) and L_{mean} in seconds (s), resulting in the energy value expressed in Joules (J).

For the Laptop CPU setup, the EPI_{cpu} was calculated as $25.0\ \text{W} \cdot 0.0844\ \text{s} = 2.11\ \text{J}$. In contrast, the Edge GPU architecture achieved an EPI_{gpu} of $6.7\ \text{W} \cdot 0.0651\ \text{s} = 0.43\ \text{J}$. This indicates that the edge-based solution is approximately 5 times more energy-efficient per unit of computation. Assuming a standard RBKairos platform with a 700 Wh battery capacity and a base system power draw of approximately 300 W (including the base mobility and the FR3 arm), reducing the perception compute draw translates to a practical increase of approximately 25 minutes of continuous autonomous operation per charge.

VII. Conclusion

This study has successfully demonstrated that an optimized edge GPU architecture is a critical enabler for autonomous mobile robots in precision agriculture. By combining hardware-accelerated visual pipelines with a mathematically robust calibration framework, we have developed a system that is both energy-efficient and highly deterministic. The integration of localized perception not only enhances the precision of robotic manipulation but also ensures the operational independence required for deployment in challenging agricultural environments.

It is important to emphasize that the primary novelty of this research does not lie in the invention of a fundamentally new artificial intelligence paradigm, but rather in the practical, end-to-end architectural integration of edge AI for a highly specific agricultural use-case. Furthermore, the performance comparison between the external laptop CPU and the edge GPU is deliberately constructed to simulate the computational constraints of the RBKairos robot's default internal processor. The objective is not to benchmark the Jetson Nano against state-of-the-art flagship GPUs, which inherently consume vastly more power, but to conclusively demonstrate that offloading perception tasks to a lightweight, highly optimized edge accelerator is strictly superior to standard CPU processing in constrained mobile robotic environments.

VIII. Extensions and Future Work

For future research, several key improvements are planned. Firstly, we intend to transition from the current open-loop execution to a closed-loop visual servoing principle. By implementing feedback control based on real-time visual updates, the system will be able to dynamically compensate for target movement or robot base vibrations during the grasp.

Secondly, the deployment on more recent hardware platforms is necessary. The NVIDIA Jetson Nano used in this work exhibits limitations in memory bandwidth for modern, large-scale transformer-based models. Upgrading to the Orin series would allow for even greater computational efficiency and the implementation of more robust inference models. Finally, we plan to design a custom integrated enclosure with an optimized active cooling system to ensure thermal stability during prolonged outdoor operations in high-temperature agricultural environments.

Acknowledgment

This work was supported by Khalifa University under Award No. HARBOT (KU UB JRP), RIG-2023-117, and KUCARS. Part of this research was conducted at the Palace of Science, Miodrag Kostic Endowment.

References

[1] A. Walter, R. Finger, R. Huber, and N. Buchmann, "Smart farming is key to developing sustainable agriculture," *PNAS*, vol. 114, no. 24, pp. 6148–6150, 2017.

[2] B. Kehoe, S. Patil, P. Abbeel, and K. Goldberg, "A survey of research on cloud robotics and automation," *IEEE Transactions on automation science and engineering*, vol. 12, no. 2, pp. 398–409, 2015.

[3] M. i. Chen, Y. Hao, X. Hu, and J. Huang, "Collaborative computation offloading for multi-uavs in mobile edge computing," *IEEE Internet of Things Journal*, vol. 8, no. 16, pp. 12 739–12 750, 2021.

[4] N. Surantha and N. Sutisna, "Key considerations for real-time object recognition on edge computing devices," *Applied Sciences*, vol. 16, 2025.

[5] V. Antoska, K. Jovanovic, V. Petrovic, N. Bascarevic, and M. Stankovski, "Balance analysis of the mobile anthropomimetic robot under disturbances – zmp approach," *International Journal of Advanced Robotic Systems*, vol. 10, no. 4, 2013.

[6] S. Alamouti, "Quantifying energy and cost benefits of hybrid edge cloud: Analysis of traditional and agentic workloads," *arXiv preprint arXiv:2501.14823*, 2025.

[7] S. Malik, N. Ruzic, M. Khonji, K. Jovanovic, J. Dias, N. Knezevic, and L. Seneviratne, "Optimizing multi-robot autonomous crop collection," in *Mechanisms and Machine Science*. Springer, 2025, pp. 696–705.

[8] L. Keselman, J. I. Iselin, P. Corey *et al.*, "Intel realsense stereoscopic depth cameras," in *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, 2017, pp. 1–10.

[9] Robotnik Automation, *Rb-kairos: Mobile manipulator for industrial applications*, https://robotnik.eu/products/mobile-robots/rb-kairos-2/#rb-kairos-plus, Robotnik Automation S.L.L., 2022.

[10] S. Haddadin, S. Parusel, L. Johannsmeier, S. Golz, S. Gabl, F. Knapp, M. Chalon, S. Jörg, T. Wieland, W. Schwarz *et al.*, "The franka emika robot: A reference platform for robotics research and education," *IEEE Robotics and Automation Magazine*, vol. 29, no. 2, pp. 46–64, 2022.

[11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.

[12] Evan-Amos, "Banana-single," https://commons.wikimedia.org/wiki/File:Banana-Single.jpg, 2012, licensed under CC BY-SA 3.0.

[13] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with cuda," *Queue*, vol. 6, no. 2, pp. 40–53, 2008.

[14] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on pattern analysis and machine intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.